JOB PULSE

DEPI Project Documentation

1. Project Overview

1.1 Project Summary

Job Pulse is a proposed data engineering project that collects, organizes, and analyzes job listings from selected online sources. It focuses on technology-related roles such as Data Engineering, Data Analysis, Data Science, Artificial Intelligence, and related careers. The project aims to transform dispersed job information into a structured dataset that can be searched, analyzed, and reported.

1.2 Problem Statement

Job opportunities are distributed across multiple websites and may be presented in inconsistent formats. Job seekers can find it difficult to compare roles, identify frequently requested skills, and understand differences in location, work mode, and experience requirements. Manual collection and comparison can also be time-consuming. Job Pulse proposes a repeatable workflow to collect permitted job data, clean it, store it, and produce useful summaries.

1.3 Proposed Solution

The system will use source-specific collectors to obtain available job data, a transformation stage to validate and standardize records, a database to store structured listings, and reporting queries or visualizations to summarize trends.

1.4 Objectives

- Collect job listings from selected sources where access is permitted and technically supported.
- Capture available fields such as title, company, location, posting date, work mode, career level, skills, and salary information.
- Clean inconsistent text, standardize selected categories, and handle missing values transparently.
- Store valid records in a structured database
- Generate reports about job counts, skill demand, locations, work arrangements, and experience levels

1.5 Scope

Collection from selected, permitted sources or datasets

Data cleaning, validation, normalization, and deduplication

Relational database storage and SQL analysis

CSV/Excel exports and analytical reports

Testing, documentation, GitHub version control, and presentation

1.6 Expected Deliverables

- Project proposal, plan
- Literature review and reference list.
- Python source code, SQL script
- Test plan, actual test results, execution screenshots, and issue log.
- Final report, presentation slides, and a well-organized GitHub repository.

Gantt-Style Schedule

Workstream	16–23 Oct	24–30 Oct	31 Oct–6 Nov	7–20 Nov	21–30 Nov	1–4 Dec
Planning and proposal	■	Review				
Literature review	■	■	Refine			
Requirements gathering	■	■	Approve			
System analysis and design		Initial	■	Refine		
Implementation			Prototype	■	■	
Testing and data validation			Plan	Initial	■	■
Reports and presentation				Outline	Draft	■

2.4 Task Assignment and Roles

Role	Main Responsibilities	Assigned Team Member
Project Manager	Coordinate schedule, meetings, milestones, task tracking, and repository progress.	Amal Amir
Data Collection	Build source connectors, extraction logic, retries, and source-level validation.	Amal Amir
Data Engineering	Clean, transform, validate, deduplicate, and load data.	Menna , Radwa
Database & Analytics	Database schema, SQL queries, data quality checks, and reports.	Amal , Heger, Radwa , Menna
Testing & Presentation Lead	Test plan, execution evidence, defect tracking, demo, and slides.	Hager

Risk Assessment and Mitigation

Risk	Mitigation / Response
Source blocks automated requests	Use permitted APIs, public datasets, or accessible sources.
Website layout changes	Keep collectors modular, add schema checks, and test selectors regularly.
Missing or inconsistent fields	Document cleaning rules, preserve unknown values, and validate required fields.
Database or network failure	Use connection checks, error handling, logging, and safe retry behavior.
Deadline slippage	Track weekly milestones, prioritize a minimum viable pipeline, and raise blockers early.
Secrets committed to GitHub	Use environment variables.
Incorrect analytical conclusions	Verify metrics with test data, document limitations, and independently check SQL results.
Unclear data redistribution rights	Check source terms and avoid uploading data that cannot legally be redistributed.

2.6 Key Performance Indicators (KPIs)

The following targets are proposed for planning. They are not measured results and should be adjusted to the selected source, dataset size, and project scope.

KPI	Measurement
Extraction success rate	Valid records collected / collection attempts, using a clearly defined denominator
Required-field completeness	Records with required fields / total records
Duplicate rate after cleaning	Remaining duplicate records / total records
Database load success	Valid records successfully loaded / valid records submitted
Report correctness	Metrics matching independently checked test results
Execution time	Elapsed time for a defined batch
Repository completeness	Completed required deliverables / agreed deliverables

3. Literature Review

3.1 Research Objective

The literature review examines established approaches to job data collection, web extraction, data integration, data quality, relational storage, ETL pipelines, and job market analytics. It should connect the selected techniques to the practical needs and limitations of Job Pulse.

3.2 Feedback, Evaluation, and Improvements

Item	What to Record
Lecturer feedback	Exact feedback, date received, requested changes, and responsible team member.
Project evaluation	Whether scope, design, implementation, and evidence satisfy the rubric.
Suggested improvement	More approved sources, improved skill normalization, incremental updates, historical trends, or better data quality checks.
Change tracking	Issue or task ID, planned change, completion date, and evidence.

3.3 Final Grading Criteria

Assessment Area	Proposed Weight
Documentation and literature review	20%
Requirements and system design	15%
Implementation and code quality	30%
Testing and validation	20%
Final presentation and demonstration	15%
Total	100%

4. Requirements Gathering

4.1 Stakeholder Analysis

Stakeholder	Needs / Expectations
Job seekers	Searchable listings, relevant filters, traceable links, and useful job information.
Project team	Reliable extraction, maintainable code, clear roles, and manageable tasks.
Lecturer / evaluator	Evidence of planning, research, design, implementation, testing, and results.
Data analyst	Consistent fields, clear definitions, and accurate summaries.
Job-source providers	Respectful and authorized access; compliance with platform terms.
Developer / administrator	Secure configuration, logs, manageable database operations, and recoverable failures.

4.2 User Stories

1. Collect listings: As a project team member, I want to collect job listings from supported sources so that I can build a centralized dataset.
2. Clean records: As a data engineer, I want to standardize fields and remove duplicates so that the data is suitable for analysis.
3. Filter jobs: As a job seeker, I want to filter listings by title, location, skills, and work mode so that I can identify relevant opportunities.
4. Analyze skills: As a job seeker or analyst, I want to see frequently requested skills so that I can understand employer requirements.
5. Monitor runs: As a developer, I want errors and run summaries to be logged so that I can troubleshoot failures.
6. Export results: As an analyst, I want to export clean results so that I can review them in a spreadsheet or reporting tool.

4.3 Use Cases

Use Case	Actor	Main Flow	Expected Result
Collect jobs	Developer / pipeline	Select source → request data → extract records	Raw records collected.
Process jobs	Pipeline	Validate → clean → normalize → deduplicate	Cleaned dataset.
Store jobs	Pipeline	Check schema → insert or update records	Database updated.
Explore jobs	Job seeker / analyst	Choose supported filters → view matching records	Relevant records returned.
Generate report	Analyst	Run queries → summarize results → export	Report produced.
Test pipeline	Developer / tester	Execute tests → inspect logs and outputs	Results and defects documented.

4.4 Functional Requirements

Requirement	Description
Source configuration	The system shall support configurable, approved job data sources.
Data extraction	The system shall collect available job attributes from each supported source.
Data validation	The system shall validate required fields such as a title and a source identifier or URL where available.
Data cleaning	The system shall standardize text, handle missing values, and normalize relevant fields.
Duplicate detection	The system shall identify likely duplicate listings using source IDs and/or normalized attributes.
Database storage	The system shall store valid processed records in structured tables.
Incremental updates	Where feasible, the system shall update existing records and insert new records without unnecessary duplicates.
Search and filtering	If a user interface is implemented, users shall be able to filter records by supported attributes.
Analytical reporting	The system shall report on job counts, skill frequency, location, work mode, and experience level where available.
Error handling	The system shall record relevant extraction, transformation, and loading errors.
Data export	The system shall support CSV or Excel export if included in the implementation.
Documentation	The repository shall include setup instructions, data definitions, execution steps, and test evidence.

4.5 Non-Functional Requirements

Category	Requirement
Performance	Process a defined test batch within a target agreed after baseline measurement.
Reliability	Handle individual record errors without silently corrupting the dataset.
Data quality	Validate required fields and document all cleaning rules.
Security	Keep credentials outside source code and use least-privilege database access.
Usability	

	Provide clear setup instructions and understandable report labels.
Maintainability	Separate collection, transformation, loading, and reporting modules.
Scalability	Allow additional sources to be integrated without rewriting the whole pipeline.
Portability	Document dependencies and configuration so the workflow can be reproduced.
Traceability	Record source and collection time, plus processing status where practical.
Compliance	Follow source terms, relevant privacy requirements, and access restrictions.